Capstone Project Report (Java-Spring Boot)

Name: Sriram M K
CSID:SMK5@ford.com
Trainer: Anand Pawar

Employee Management System:

This is a Employee Management web application built with Java, Spring Boot, Spring Security, Thymeleaf, and Maven. It provides secure authentication, allowing users to log in, view all employees, create new employees via a form, and raise employee salaries. The UI is rendered using Thymeleaf templates, and all data is managed through standard Spring MVC and JPA layers with data stored in a Postgres DB.

1.Login Page:

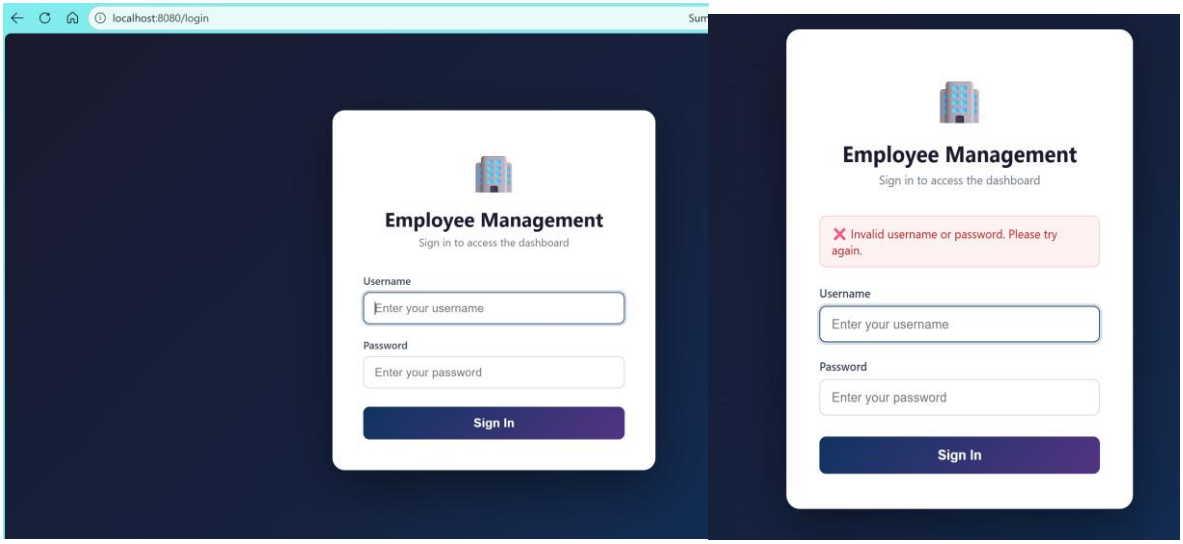


Fig 1: Login Page

The login page provides a secure entry point for users to access the Employee Management application. It displays a simple form where users enter their username and password.

When the user submits the form, Spring Security intercepts the request, authenticates the credentials, and, if valid, grants access to the application. If authentication fails, the user is shown an error message and prompted to try again. The login page is rendered using a Thymeleaf template.

2.HomePage:

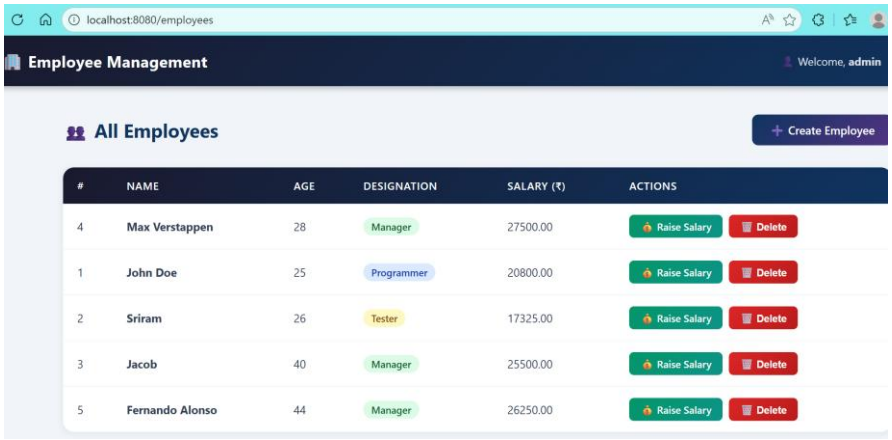


Fig 2:Employee Dashboard

The employees page displays a list of all employees in a tabular format. It provides options to create a new employee using a form and to raise the salary of existing employees. The page is rendered using a Thymeleaf template and is accessible only after successful login.

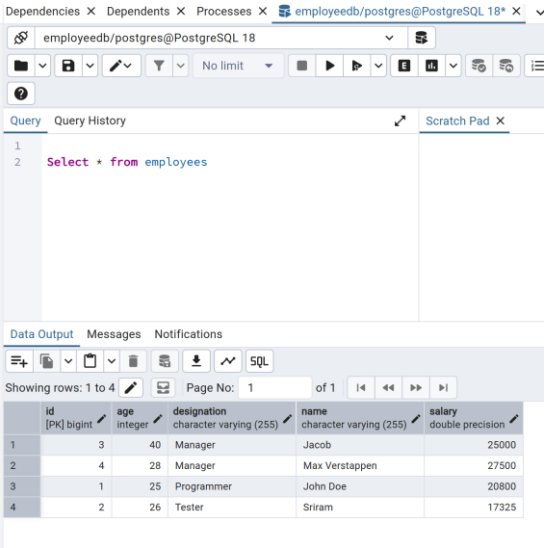


Fig 3: Database for employees

When a user navigates to /employees, the controller fetches all employee records from the database, adds them to the model, and returns the employees view. Thymeleaf uses this data to render the employee list and action buttons on the page. Access is protected by Spring Security, ensuring only authenticated users can view this page.

3.Create Employee:

The create-employee page provides a form for adding a new employee to the system. Users can enter details such as name and salary. The page is rendered using the create-employee.html Thymeleaf template and is accessible only to authenticated users.

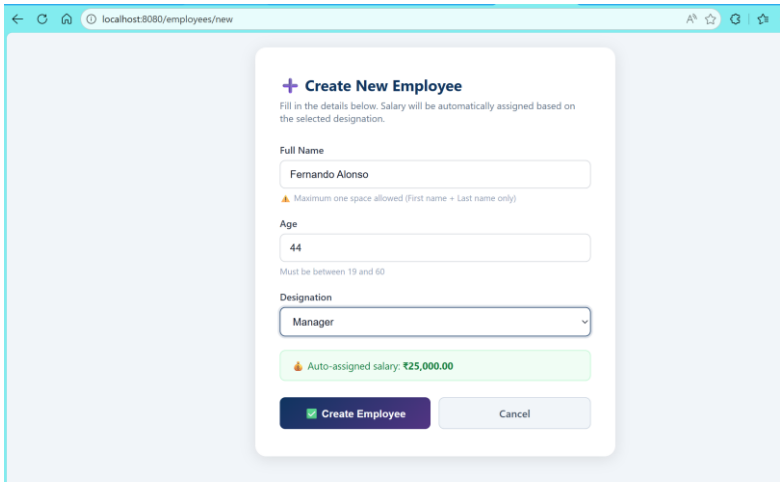


Fig 4: Creating Employee Form

Logic for /employees/new:

GET /employees/new: Displays a blank form for entering new employee details. The controller adds an empty EmployeeRequestDTO to the model for form binding.

POST /employees/new: Handles form submission. The controller validates and saves the new employee using the service layer. On success, it redirects to /employees with a success message; on error, it re-renders the form with an error message. Access is protected by Spring Security.

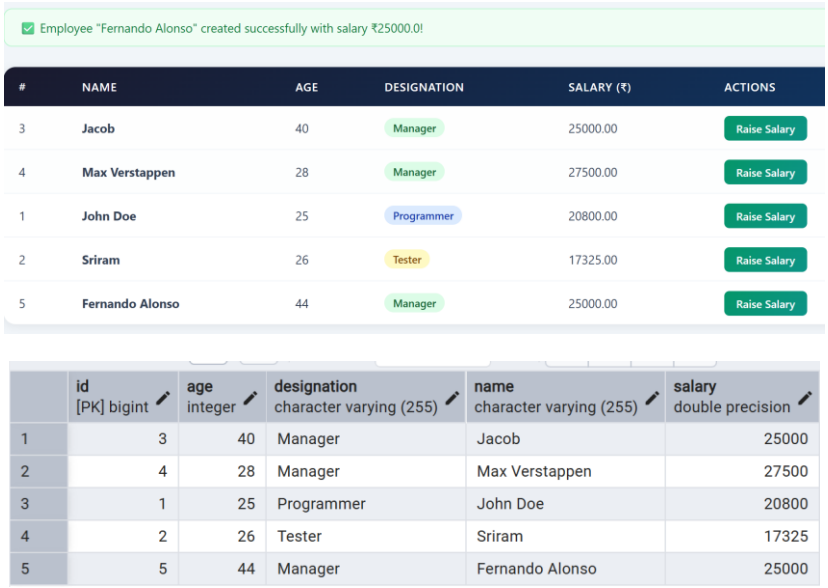
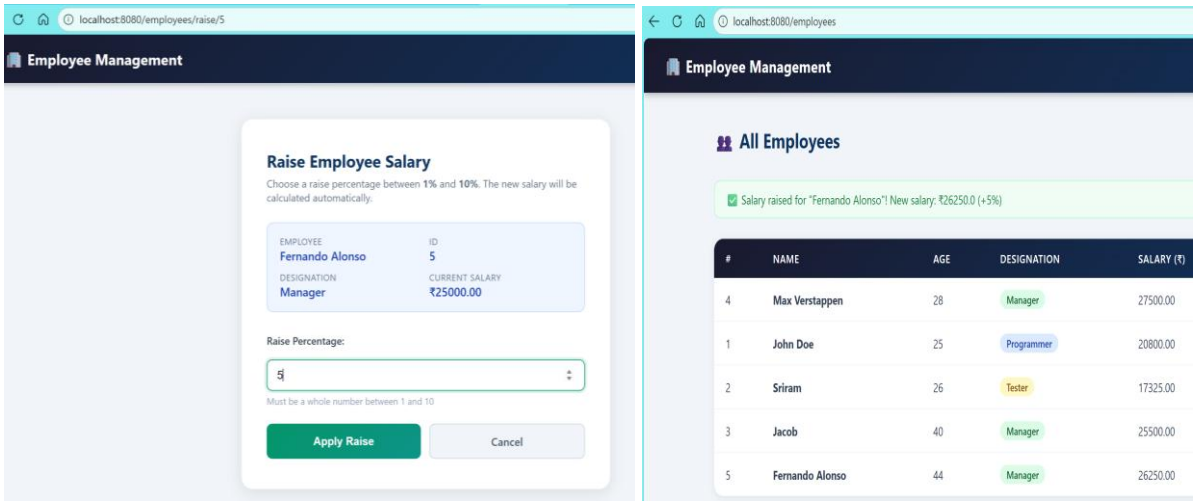


Fig 5: Employee added successfully

4.Raise Employee Salary:

The raise-salary page provides a form for increasing the salary of a specific employee. It displays the employee's current details and allows the user to enter the raise percentage. The page is rendered using the raise-salary.html Thymeleaf template and is accessible only to authenticated users.



	id [PK] bigint	age integer	designation character varying (255)	name character varying (255)	salary double precision
1	4	28	Manager	Max Verstappen	27500
2	1	25	Programmer	John Doe	20800
3	2	26	Tester	Sriram	17325
4	3	40	Manager	Jacob	25500
5	5	44	Manager	Fernando Alonso	26250

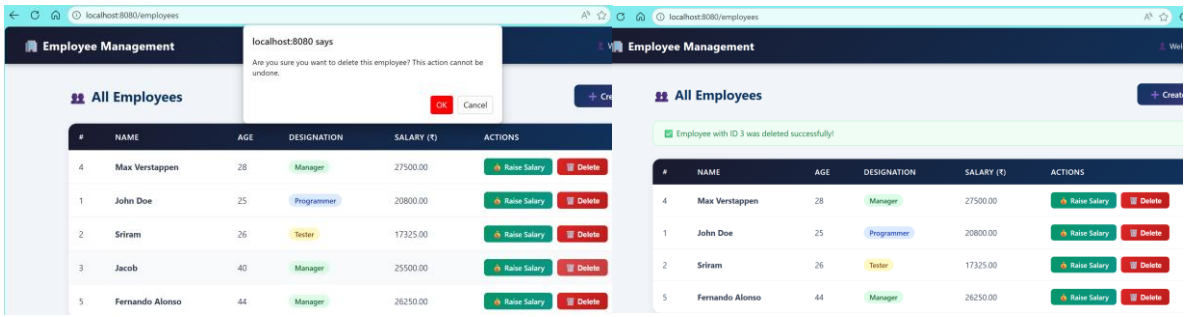
Fig 6:Salary Updated by EmpId and reflected in DB

Logic for /employees/raise/{id}:

GET /employees/raise/{id}: Fetches the employee's details by ID and displays the raise salary form. If the employee is not found, an error message is shown.

POST /employees/raise/{id}: Processes the submitted raise percentage. The controller validates and applies the salary increase using the service layer. On success, it redirects to /employees with a success message; on error, it re-renders the form with an error message. Access is protected by Spring Security.

5.Delete Employee:



	id [PK] bigint	age integer	designation character varying (255)	name character varying (255)	salary double precision
1	4	28	Manager	Max Verstappen	27500
2	1	25	Programmer	John Doe	20800
3	2	26	Tester	Sriram	17325
4	5	44	Manager	Fernando Alonso	26250

Fig 8: Delete Employee by id and reflected in DB

The delete employee feature allows authenticated users to permanently remove an employee record from the system directly from the employees list page.

Each employee row has a Delete button that submits a POST form to /employees/delete/{id}.A browser confirm() dialog asks for confirmation before deletion.

On submit, the backend deletes the employee and redirects to /employees with a success or error message.

6.Logout Page:

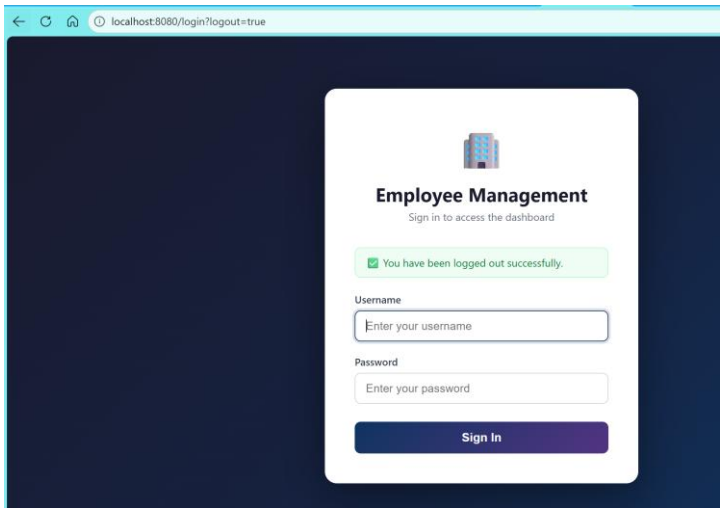


Fig 8: Redirected to Login page after clicking logout

The logout logic securely ends the user session in the Employee Management application. When a user clicks the logout button or accesses the /logout endpoint, Spring Security invalidates the session and clears authentication data. The user is then redirected to the login page with a logout success message, ensuring that protected resources remain inaccessible until the user logs in again.

GITHUB LINK:

https://github.com/srirepo-05/smk5-java-training/tree/main/EmployeeManagement